University of Prince Edward Island

Machine Learning

Decision Trees



Decision Trees Learning Method

- Decision tree learning is a method for approximating discrete-valued target functions, in which the learned function is represented by a decision tree.
- Learned trees can also be represented as sets of if-then rules to improve human readability.
- These learning methods are among the most popular of inductive inference algorithms and have been successfully applied to a broad range of tasks from learning to diagnose medical cases to learning to assess credit risk of loan applicants.

Appropriate Problems for Decision Trees

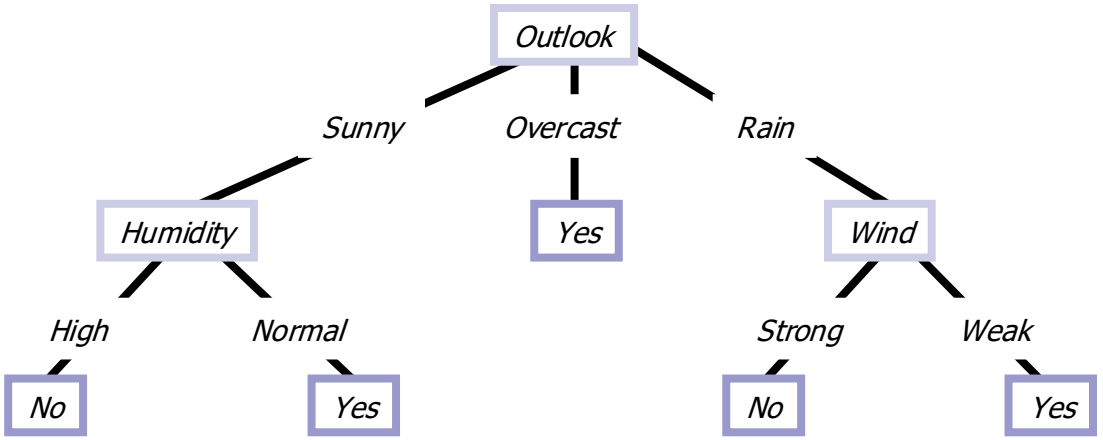
- **Instances are represented by attribute-value pairs.** Instances are described by a fixed set of attributes (e.g., **Temperature)** and their values (e.g., **Hot).**
- **The target function has discrete output values.** A substantial extension allows learning target functions with real-valued outputs, though the application of decision trees in this setting is less common.
- **Disjunctive descriptions may be required.** Decision trees naturally represent disjunctive expressions.
- **The training data may contain errors or missing attribute values.** Decision trees are robust to errors, both errors in classifications of the training examples and errors in the attribute values that describe these examples.

An Example – Predicting Tennis

Example	Attributes				Target
	Outlook	**Temp.**	**Humidity**	**Wind**	**Play Tennis**
D1	Sunny	Hot	High	Weak	NO
D2	Sunny	Hot	High	Strong	NO
D3	Overcast	Hot	High	Weak	YES
D4	Rain	Mild	High	Weak	YES
D5	Rain	Cool	Normal	Weak	YES
D6	Rain	Cool	Normal	Strong	NO
D7	Overcast	Cool	Normal	Weak	YES
D8	Sunny	Mild	High	Weak	NO
D9	Sunny	Cool	Normal	Weak	YES
D10	Rain	Mild	Normal	Strong	YES
D11	Sunny	Mild	Normal	Strong	YES
D12	Overcast	Mild	High	Strong	YES
D13	Overcast	Hot	Normal	Weak	YES
D14	Rain	Mild	High	Strong	NO

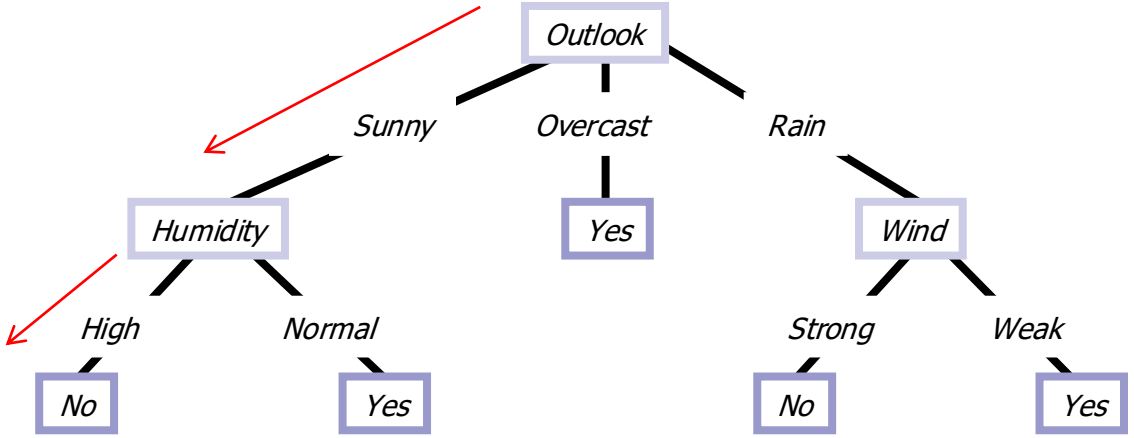
Decision Tree Representation

Decision trees classify instances by sorting them down the tree from the root to some leaf node, which provides the classification of the instance. Each node in the tree specifies a test of some attribute of the instance, and each branch descending from that node corresponds to one of the possible values for this attribute.



Decision Tree Representation

Decision trees classify instances by sorting them down the tree from the root to some leaf node, which provides the classification of the instance. Each node in the tree specifies a test of some attribute of the instance, and each branch descending from that node corresponds to one of the possible values for this attribute.



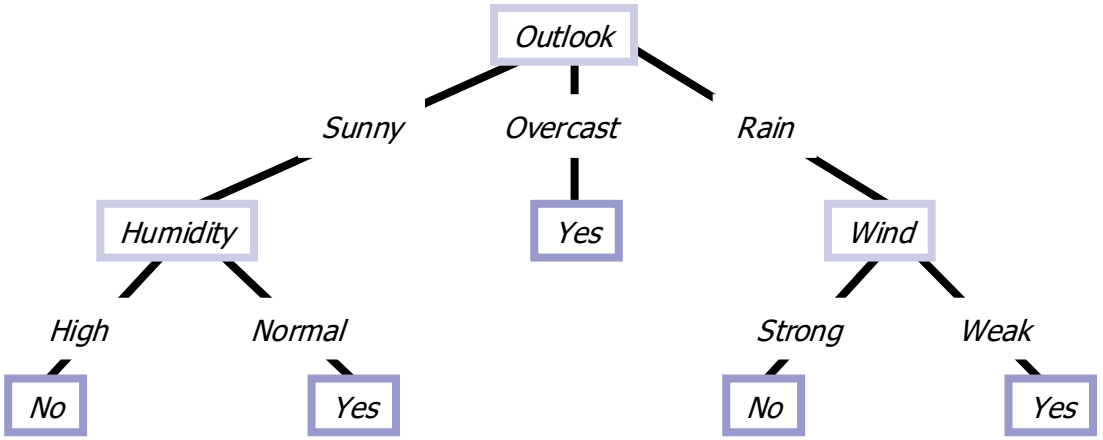
Given this new instance:

(***Outlook*** = *Sunny*,
Temperature = *Hot*,
Humidity = *High*,
Wind = *Strong*)

Prediction is: **No** tennis

Decision Tree Representation

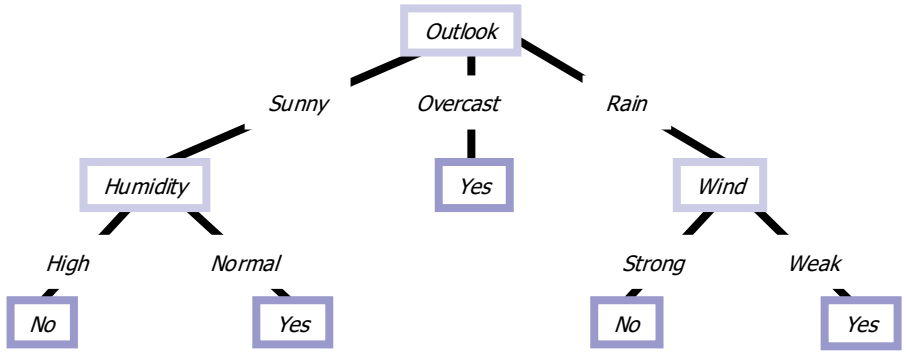
Decision trees classify instances by sorting them down the tree from the root to some leaf node, which provides the classification of the instance. Each node in the tree specifies a test of some attribute of the instance, and each branch descending from that node corresponds to one of the possible values for this attribute.



- Decision trees represent a disjunction of conjunctions of constraints on the attribute values of instances.
- Each path from the tree root to a leaf corresponds to a conjunction of attribute tests, and the tree itself to a disjunction of these conjunctions.

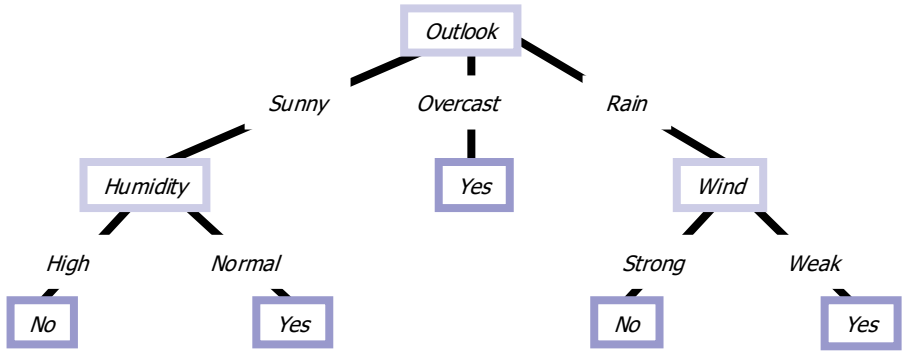
$$(Outlook=Sunny \wedge Humidity=Normal)$$
$$\vee \quad (Outlook=Overcast)$$
$$\vee \quad (Outlook=Rain \wedge Wind=Weak)$$

Choosing Attributes



- This example has 4 attributes: outlook, humidity, temperature and wind
- But the decision tree only showed 3 attributes: outlook humidity, and wind
- Why is that?

Choosing Attributes



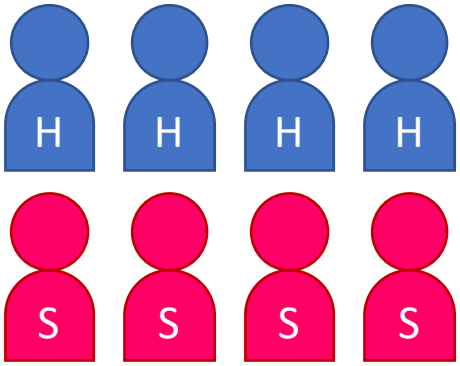
- This example has 4 attributes: outlook, humidity, temperature and wind
- But the decision tree only showed 3 attributes: outlook humidity, and wind
- Why is that?

- Methods for selecting attributes (which will describe next) show that temperature is not a discriminating attribute

- Decision Trees have a bias towards creating smaller (simpler) trees. Also known as principle of Occam's Razor: Given a number of competing hypotheses, the simplest one is preferable

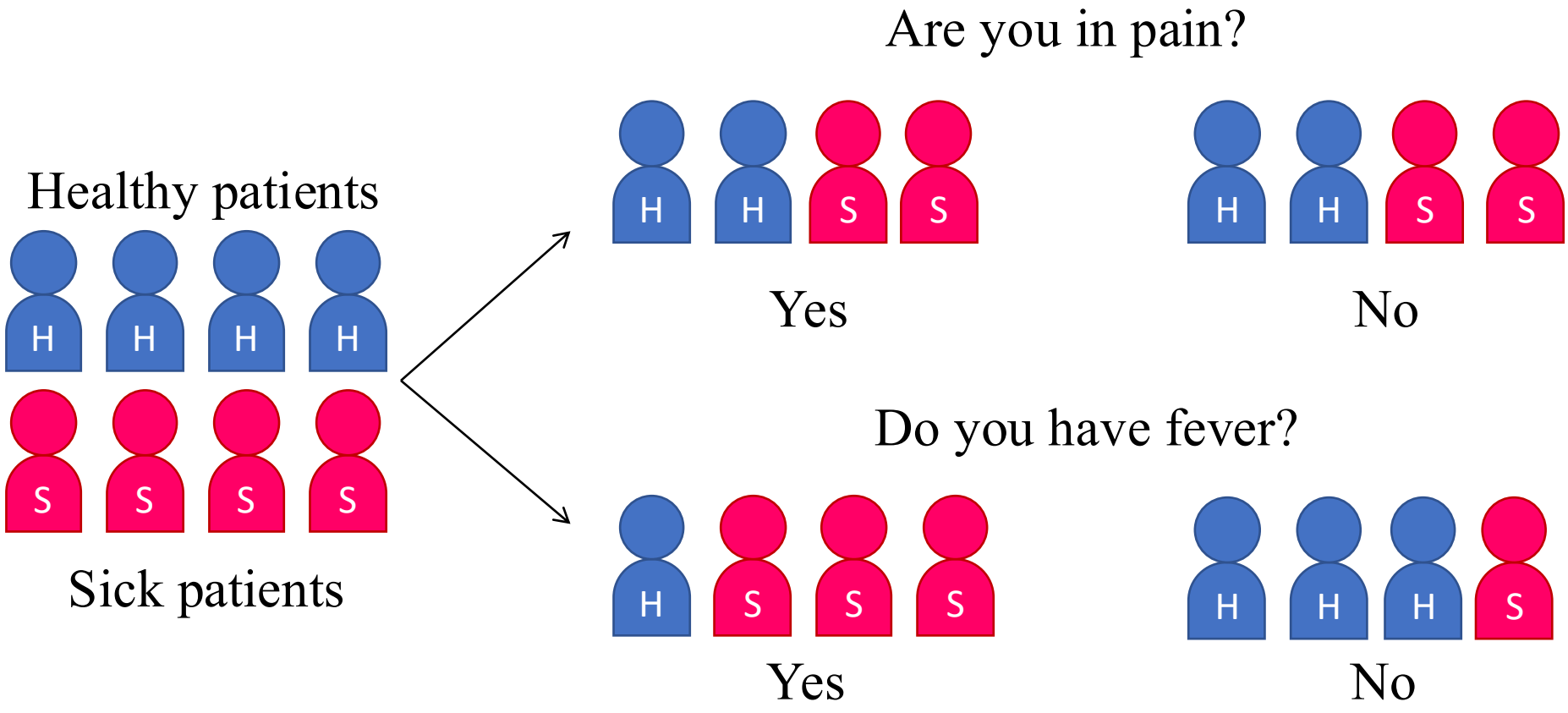
How to Select an Attribute?

Healthy patients

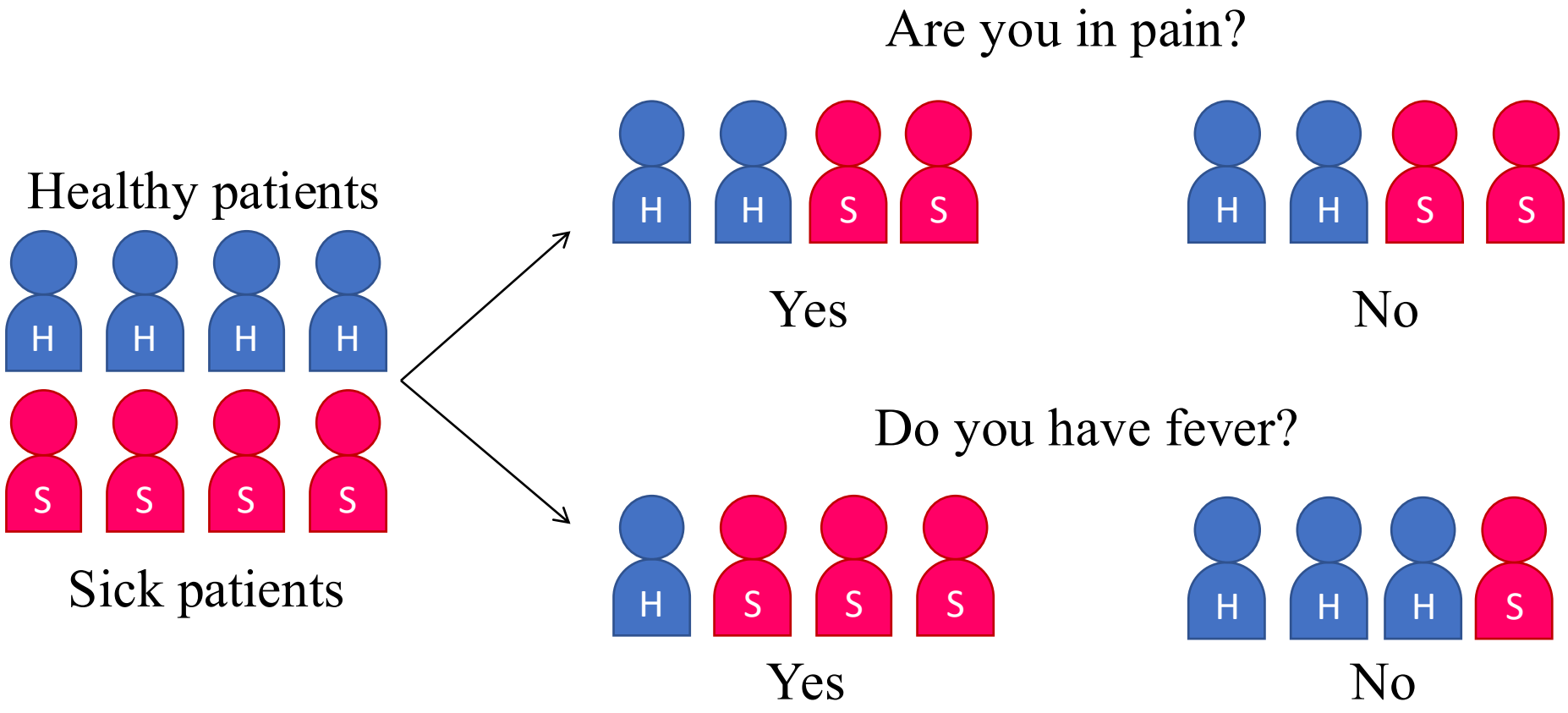


Sick patients

How to Select an Attribute?



How to Select an Attribute?



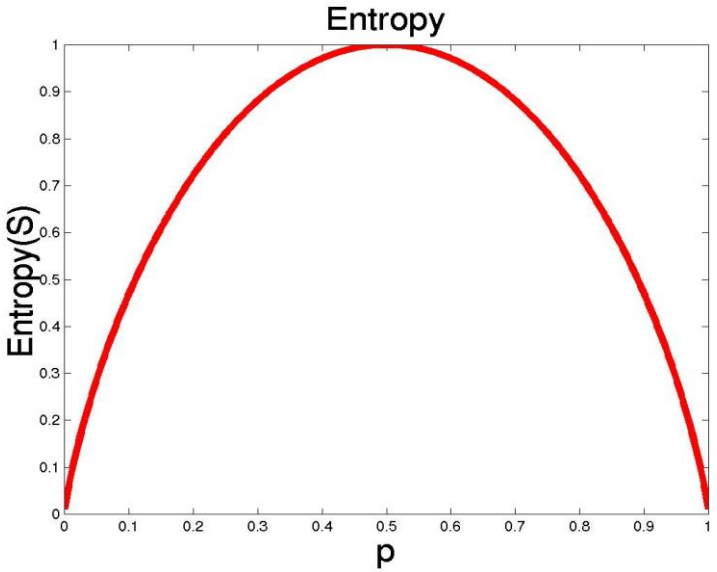
We would like to select the attribute that is most useful for classifying examples. What is a good quantitative measure of the worth of an attribute? We will define a statistical property, called **information gain,** that measures how well a given attribute separates the training examples according to their target classification.

Entropy

Entropy *characterizes the (im)purity of an arbitrary collection of examples. Given a collection S, containing positive and negative examples of some target concept, the entropy of S relative to this boolean classification is:*

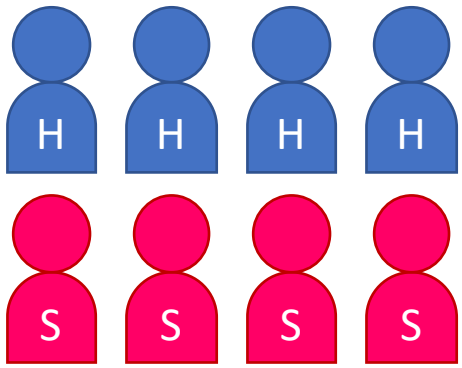
Proportion of positive examples in S

$$Entropy(S) = -p_{\oplus} \log_2(p_{\oplus}) - p_{\ominus} \log_2(p_{\ominus})$$



The entropy function relative to a boolean classification, as the proportion, $p_{\oplus}$ of positive examples varies between *0* and 1.

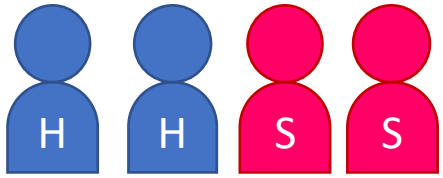
How to Select an Attribute?



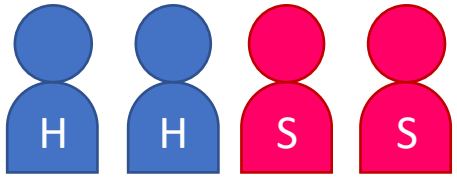
Healthy patients

Sick patients

Are you in pain?



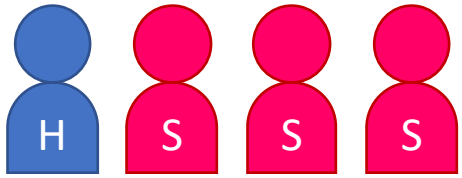
Yes



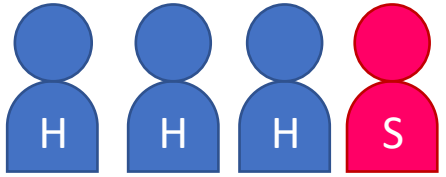
No

$$E(S_{pain}) = -(\tfrac{2}{4})\log_2(\tfrac{2}{4}) - (\tfrac{2}{4})\log_2(\tfrac{2}{4})$$
$$= -\left(\tfrac{2}{4}\right)(-1) - \left(\tfrac{2}{4}\right)(-1) = 1$$

Do you have fever?



Yes



No

$$E(S_{fever}) = -\left(\tfrac{3}{4}\right)\log_2\left(\tfrac{3}{4}\right) - \left(\tfrac{1}{4}\right)\log_2\left(\tfrac{1}{4}\right) = 0.8113$$

What if all the elements are from the same class?

Entropy

Entropy *characterizes the (im)purity of an arbitrary collection of examples. Given a collection S, containing positive and negative examples of some target concept, the entropy of S relative to this boolean classification is*

$$Entropy(S) = -p_{\oplus}\log_2(p_{\oplus}) - p_{\ominus}\log_2(p_{\ominus})$$

Entropy *can be generalized for any number of* ***c*** *classes:*

$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Information Gain

We can now define a measure of the effectiveness of an attribute in classifying the training data. The measure we will use, called **information gain,** is simply the expected reduction in entropy caused by partitioning the examples according to this attribute. More precisely, the information gain, **Gain(S, A)** of an attribute **A,** relative to a collection of examples **S,** is defined as

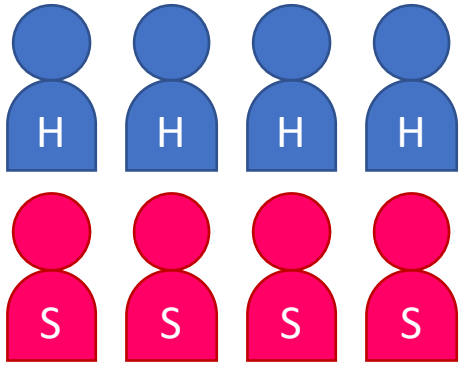
$$G(S,A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \cdot Entropy(S_v)$$

Set of all possible values of attribute A

How to Select an Attribute?

$E(S) = 1$

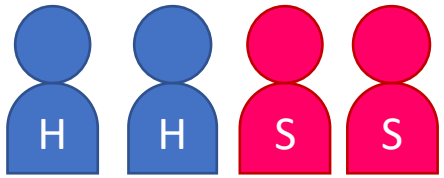
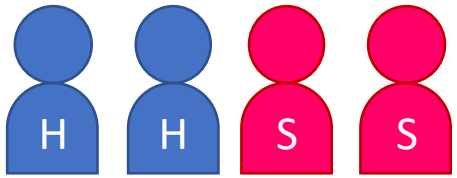
Healthy patients



Sick patients

Are you in pain?

$E(S_{Pyes}) = 1$ $E(S_{Pno}) = 1$

Yes  No 

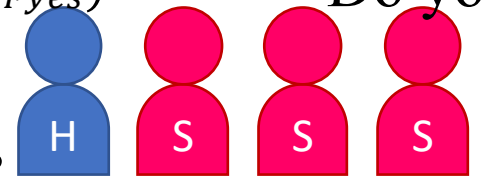
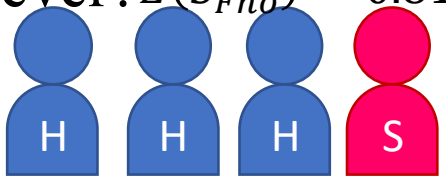
$$G(S,P) = E(S) - \sum_{v\in\{yes,no\}} \frac{|S_v|}{|S|} \cdot E(S_v)$$

$$= E(S) - \left(\frac{2}{4}\right) E(S_{Pyes}) - \left(\frac{2}{4}\right) E(S_{Pno}) = 1 - \frac{1}{2} - \frac{1}{2}$$

$$= \mathbf{0}$$

Do you have fever?

$E(S_{Fyes}) = 0.8113$ $E(S_{Fno}) = 0.8113$

Yes  No 

$$G(S,F) = E(S) - \sum_{v\in\{yes,no\}} \frac{|S_v|}{|S|} \cdot E(S_v) = E(S) - \left(\frac{3}{4}\right) E(S_{Fyes}) - \left(\frac{1}{4}\right) E(S_{Fno})$$

$$= 1 - \frac{3}{4}(0.8113) - \frac{1}{4}(0.8113) = \mathbf{0.1887}$$

An Example – Predicting Tennis

Example	Attributes				Target
	Outlook	Temp.	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	NO
D2	Sunny	Hot	High	Strong	NO
D3	Overcast	Hot	High	Weak	YES
D4	Rain	Mild	High	Weak	YES
D5	Rain	Cool	Normal	Weak	YES
D6	Rain	Cool	Normal	Strong	NO
D7	Overcast	Cool	Normal	Weak	YES
D8	Sunny	Mild	High	Weak	NO
D9	Sunny	Cool	Normal	Weak	YES
D10	Rain	Mild	Normal	Strong	YES
D11	Sunny	Mild	Normal	Strong	YES
D12	Overcast	Mild	High	Strong	YES
D13	Overcast	Hot	Normal	Weak	YES
D14	Rain	Mild	High	Strong	NO

Selecting the Next Attribute

- *Read the notation: S = [9+, 5−] means 9 YES and 5 NO.*
- *Compute the entropy of the full training set.*
- *Compute the weighted entropy after each candidate split.*
- *Choose the attribute with the largest information gain.*

Key notation

$S = [9+,\ 5-]$

$E(S) = 0.940$

$Gain(S, Outlook) = 0.247$

- *[9+, 5−] = counts in this node*
- *E(S) = entropy (impurity) of the node*
- *Gain(S, A) = reduction in entropy after splitting on attribute A*

Selecting the Next Attribute

Full set S = [9+, 5−]

YES = 9

NO = 5

$$E(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$$
$$= -(0.643)\log_2(0.643) - (0.357)\log_2(0.357)$$
$$\approx 0.940$$

- *Entropy measures how mixed the target labels are.*
- *If a node is pure, entropy = 0.*
- *The root is somewhat mixed, so entropy is positive.*
- *Now we test candidate attributes to reduce that impurity.*

Selecting the Next Attribute

Humidity = High

Examples: D1, D2, D3, D4, D8, D12, D14
Counts: [3+, 4−]
E = −(3/7)log2(3/7) − (4/7)log2(4/7) = 0.985

Humidity = Normal

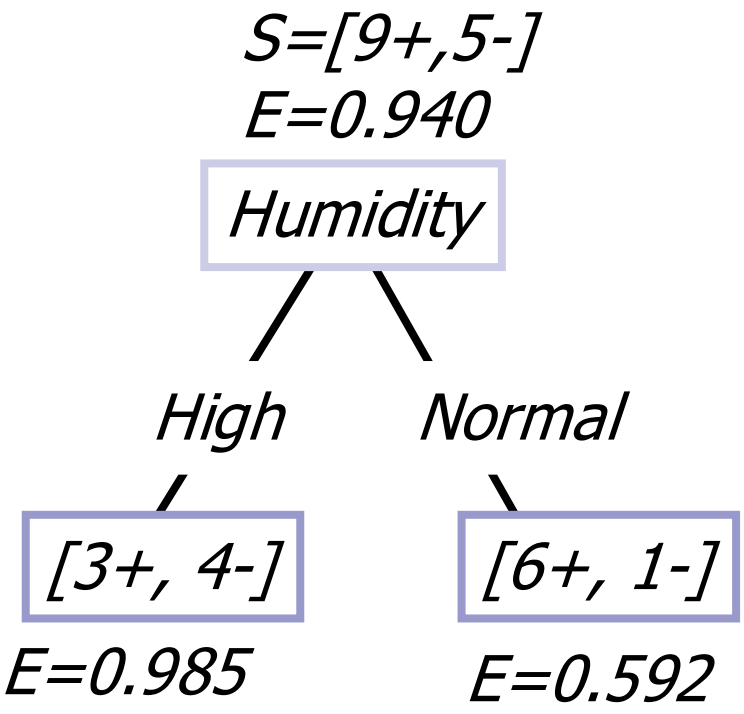
Examples: D5, D6, D7, D9, D10, D11, D13
Counts: [6+, 1−]
E = −(6/7)log2(6/7) − (1/7)log2(1/7) = 0.592

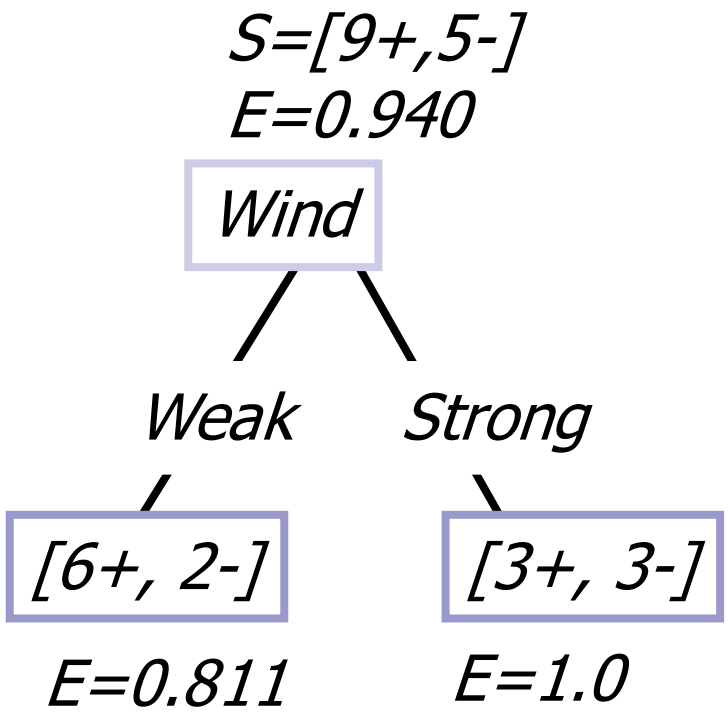
Gain(S, Humidity) = E(S) − (7/14)·0.985 − (7/14)·0.592
= 0.940 − 0.4925 − 0.2960
= 0.1515 ≈ 0.151

$$G(S,A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \cdot Entropy(S_v)$$

Notice the weights 7/14 and 7/14: they come from the branch sizes, not from the class proportions inside a branch.

Selecting the Next Attribute



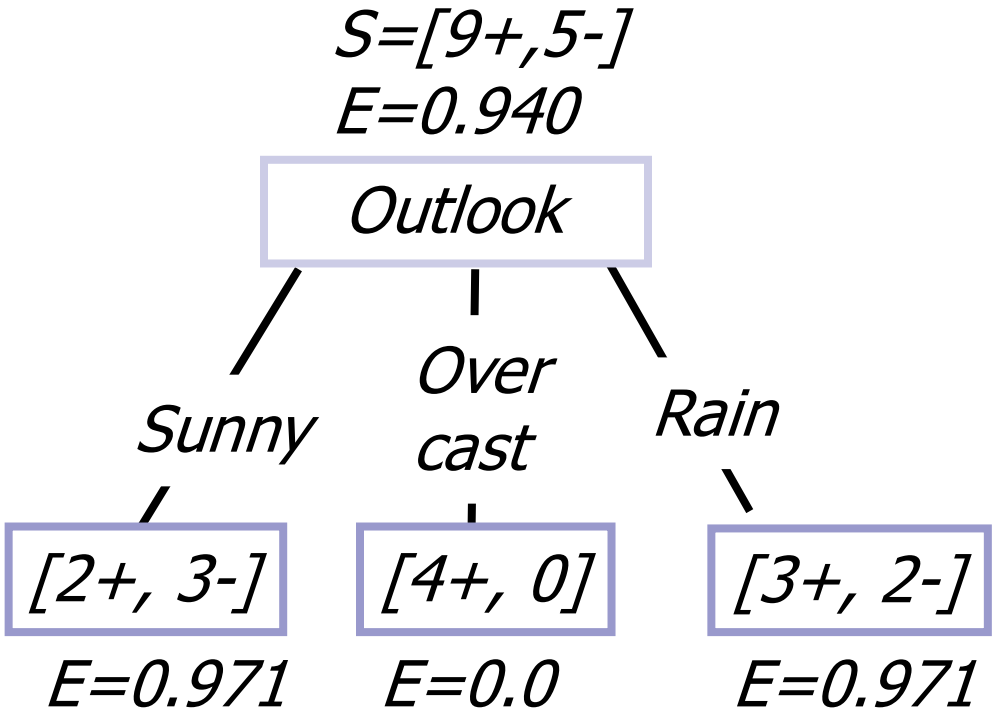


Gain(S,Humidity)
=0.940 – (7/14)*0.985
– (7/14)*0.592
=0.151

Gain(S,Wind)
=0.940 – (8/14)*0.811
– (6/14)*1.0
=0.048

Humidity provides greater info. gain than Wind, w.r.t target classification.

Selecting the Next Attribute



Gain(S,Outlook)
=0.940-(5/14)*0.971
-(4/14)*0.0 – (5/14)*0.971
=0.247

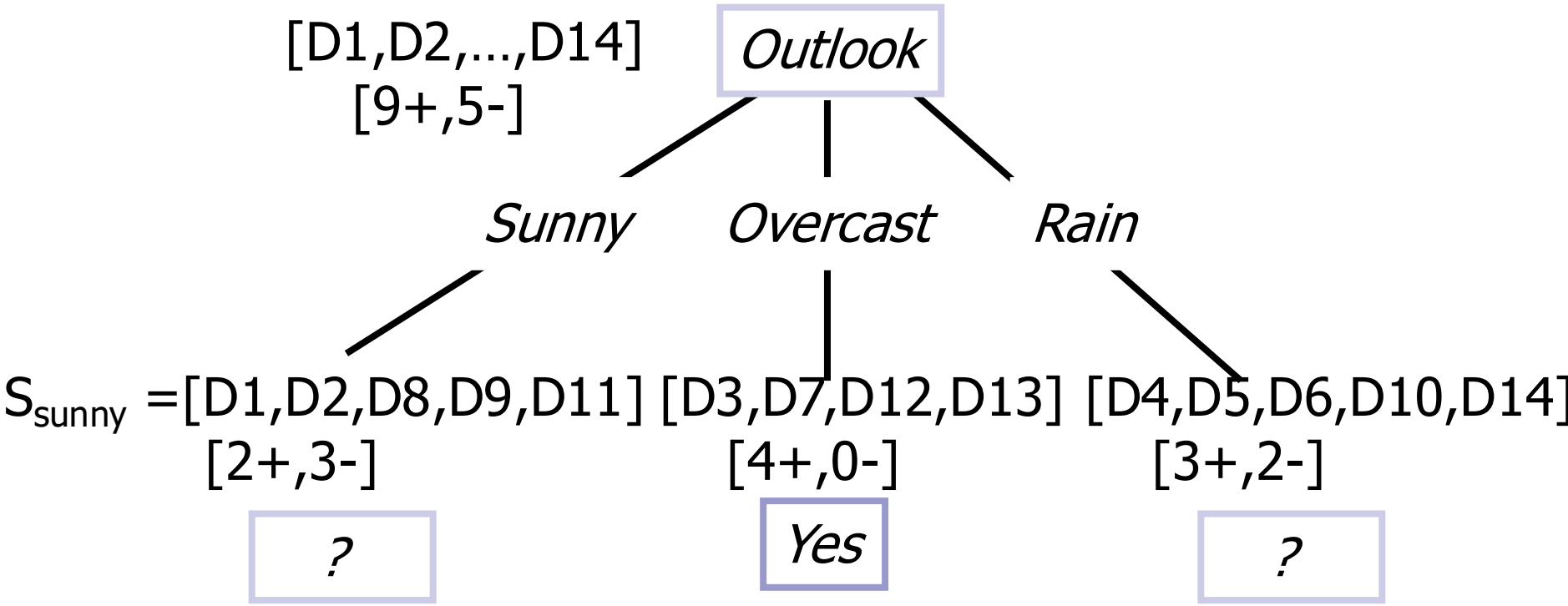
Selecting the Next Attribute

The information gain values for the 4 attributes are:

- Gain(S,Outlook) =0.247
- Gain(S,Humidity) =0.151
- Gain(S,Wind) =0.048
- Gain(S,Temperature) =0.029

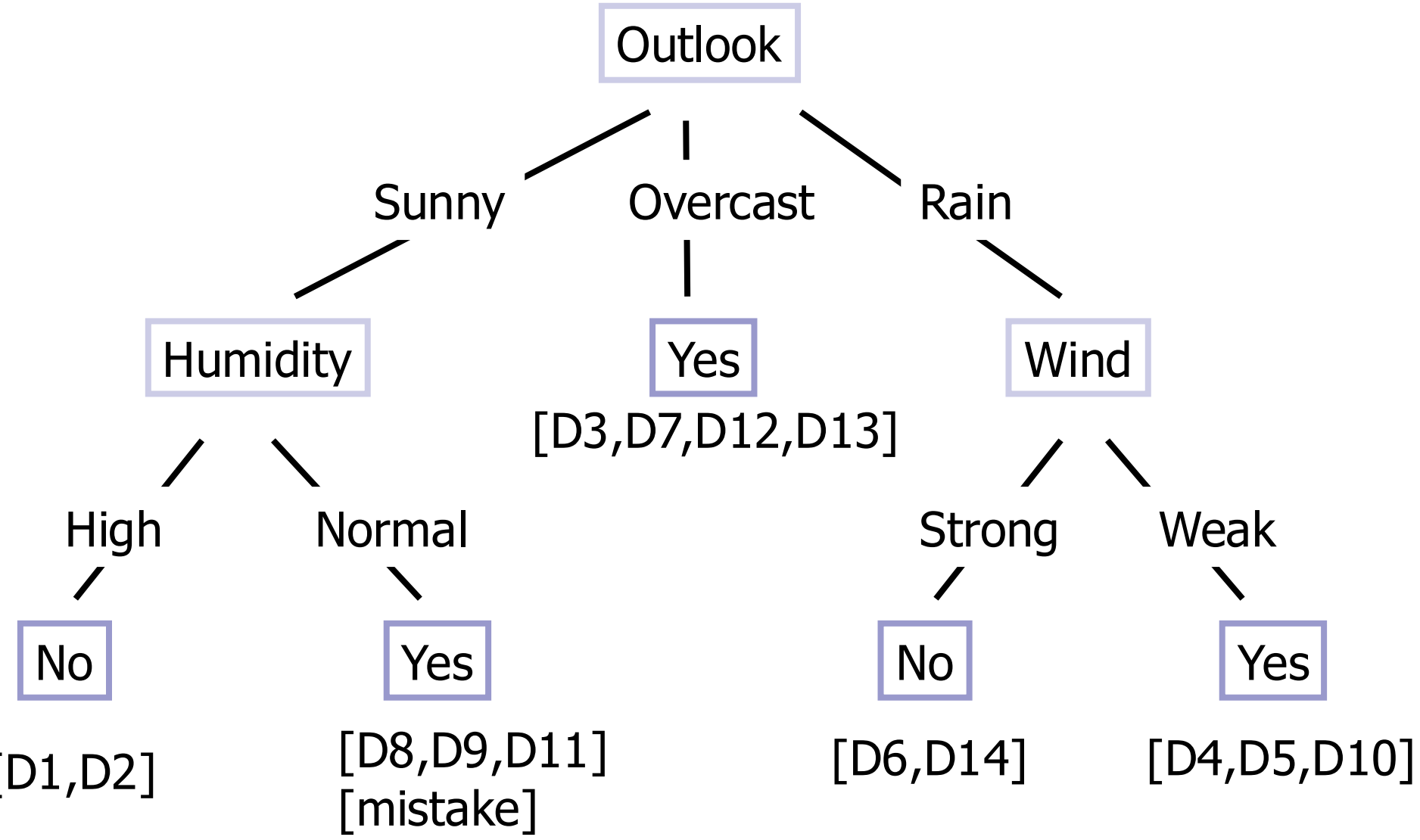
where S denotes the collection of training examples

ID3 Algorithm

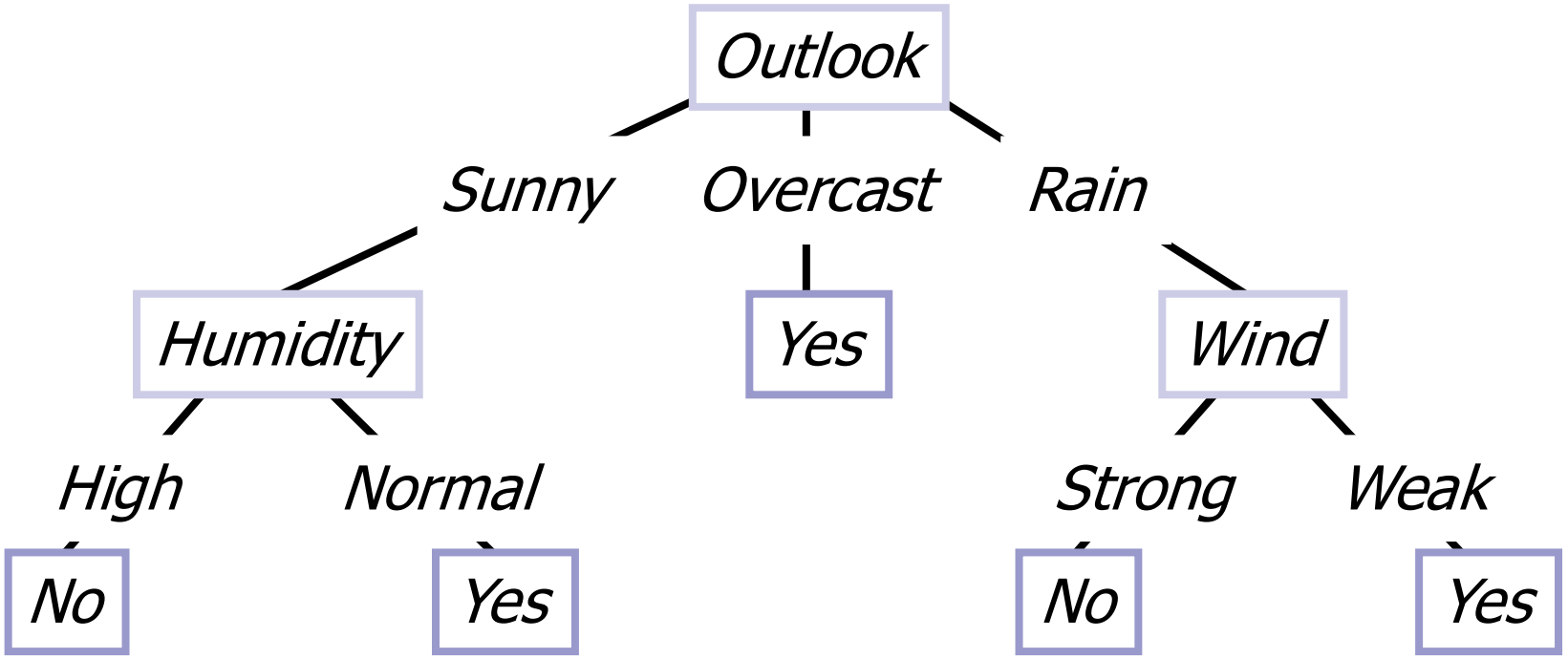


Gain(S_{sunny}, Humidity)=0.970-(3/5)0.0 – 2/5(0.0) = 0.970
Gain(S_{sunny}, Temp.)=0.970-(2/5)0.0 –2/5(1.0)-(1/5)0.0 = 0.570
Gain(S_{sunny}, Wind)=0.970= -(2/5)1.0 – 3/5(0.918) = 0.019

ID3 Algorithm



Converting a Tree to Rules



R_1: If (Outlook=Sunny) $\wedge$ (Humidity=High) Then PlayTennis=No
R_2: If (Outlook=Sunny) $\wedge$ (Humidity=Normal) Then PlayTennis=Yes
R_3: If (Outlook=Overcast) Then PlayTennis=Yes
R_4: If (Outlook=Rain) $\wedge$ (Wind=Strong) Then PlayTennis=No
R_5: If (Outlook=Rain) $\wedge$ (Wind=Weak) Then PlayTennis=Yes

Iterative Dichotomiser (ID3) Algorithm

```
ID3 (Examples, Target_Attribute, Attributes)
   Create a root node for the tree
   If all examples are positive, Return the single-node tree Root, with label = +.
   If all examples are negative, Return the single-node tree Root, with label = -.
   If number of predicting attributes is empty, then Return the single node tree Root,
    with label = most common value of the target attribute in the examples.

   Otherwise Begin
      A ← The Attribute that best classifies examples.
      Decision Tree attribute for Root = A.

      For each possible value, v_i, of A,

         Add a new tree branch below Root, corresponding to the test A = v_i.

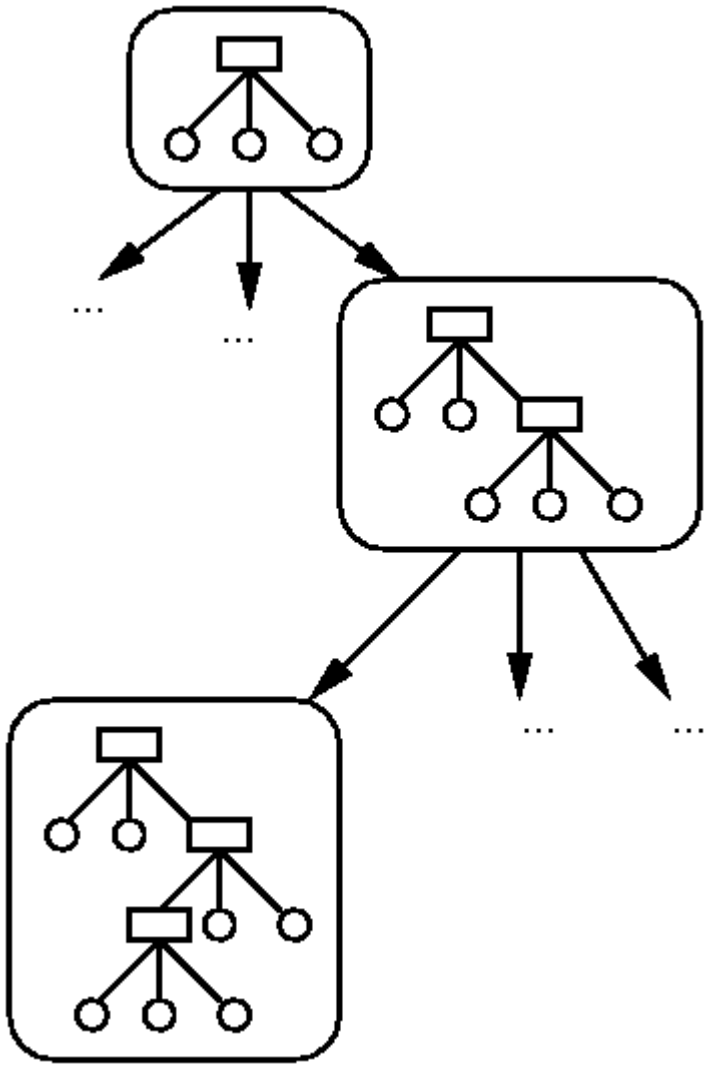
         Let Examples(v_i) be the subset of examples that have the value v_i for A

         If Examples(v_i) is empty
            Then below this new branch add a leaf node with label = most common
                    target value in the examples

         Else below this new branch add the subtree ID3 (Examples(v_i), Target_Attribute,
          Attributes - {A})
   End
   Return Root
```

1. Calculate the entropy of every attribute of the data set.
2. Split the set into subsets using the attribute that maximizes the information gain.
3. Make a decision tree node containing that attribute.
4. Recurse on subsets using the remaining attributes.

ID3 – As a Search Algorithm



1. Searches over the entire hypothesis space.
2. Uses a greedy strategy by selecting the locally best attribute to split the dataset on each iteration.
3. Does not guarantee an optimal solution. It can converge upon local optima.
4. Maintains only a single current hypothesis as it searches through the space of decision trees.
5. Inductive bias: “prefers shortest tree”

Another Example – Predicting Commute Time

Example	Attributes				Target
	Hour	Weather	Accident	Stall	Commute
D1	8 AM	Sunny	No	No	Long
D2	8 AM	Cloudy	No	Yes	Long
D3	10 AM	Sunny	No	No	Short
D4	9 AM	Rainy	Yes	No	Long
D5	9 AM	Sunny	Yes	Yes	Long
D6	10 AM	Sunny	No	No	Short
D7	10 AM	Cloudy	No	No	Short
D8	9 AM	Rainy	No	No	Medium
D9	9 AM	Sunny	Yes	No	Long
D10	10 AM	Cloudy	Yes	Yes	Long
D11	10 AM	Rainy	No	No	Short
D12	8 AM	Cloudy	Yes	No	Long
D13	9 AM	Sunny	No	No	Medium

Another Example – Predicting Commute Time

Example	Attributes				Target
	Hour	Weather	Accident	Stall	Commute
D1	8 AM	Sunny	No	No	Long
D2	8 AM	Cloudy	No	Yes	Long
D3	10 AM	Sunny	No	No	Short
D4	9 AM	Rainy	Yes	No	Long
D5	9 AM	Sunny	Yes	Yes	Long
D6	10 AM	Sunny	No	No	Short
D7	10 AM	Cloudy	No	No	Short
D8	9 AM	Rainy	No	No	Medium
D9	9 AM	Sunny	Yes	No	Long

Which attribute is ID3 going to choose first?
What is the final tree?

Other Trees Algorithms (C4.5 and CART)

C4.5 made a number of improvements to ID3. Some of these are:

- Handling both continuous and discrete attributes.
- Handling training data with missing attribute.
- Pruning trees after creation.
- …

Continuous-Valued Attributes

- Create a new boolean attribute A, that is true if A < ***c*** and false otherwise.
- The question is how to select the best value for the threshold ***c***.

Temperature	40°C	48°C	60°C	72°C	80°C	90°C
PlayTennis	*No*	*No*	*Yes*	*Yes*	*Yes*	*No*

- Pick a threshold ***c*** that produces the greatest information gain.
- Sort the examples according to the continuous attribute **A,** then identifying adjacent examples that differ in their target classification. ((48+60)/2 and (80+90)/2)
- The information gain can then be computed for each the best can be selected (Temperature>54)

Missing Values

What if some examples have missing values of A?

- If node n tests A, assign most common value of A among other examples sorted to node n.

- Assign most common value of A among other examples with same target value

- Assign probability p_i to each possible value v_i of A
  - Assign fraction p_i of example to each descendant in tree

Alternative Measures for Selecting Attributes

There is a natural bias in the information gain measure that favors attributes with many values over those with few values.

- As an example, consider the attribute Date, which has a very large number of possible values
  - If we add this attribute to the data, it would have the highest information gain of any of the attributes.
  - Because Date alone perfectly predicts the target attribute over the training data.
  - It would be selected as the decision attribute for the root node of the tree and lead to a (quite broad) tree of depth one, which perfectly classifies the training data.
  - This decision tree would perform poorly on subsequent examples, because it is not a useful predictor despite the fact that it perfectly separates the training data.

Alternative Measures for Selecting Attributes

One way to avoid this difficulty is to select decision attributes based on some measure other than information gain. One alternative measure that has been used successfully is the gain ratio.

The gain ratio measure penalizes attributes such as *Date* by incorporating a term, called split information, that is sensitive to how broadly and uniformly the attribute splits the data:

$$SplitInformation(S, A) = -\sum_{i=1}^{c} \frac{|S_i|}{|S|} log\left(\frac{|S_i|}{|S|}\right)$$

where $\boldsymbol{S_i}$ through $\boldsymbol{S}$, are the $\boldsymbol{c}$ subsets of examples resulting from partitioning $\boldsymbol{S}$ by the c-valued attribute $\boldsymbol{A}$.

Note that Split-information is actually the entropy of S with respect to the values of attribute A.

Alternative Measures for Selecting Attributes

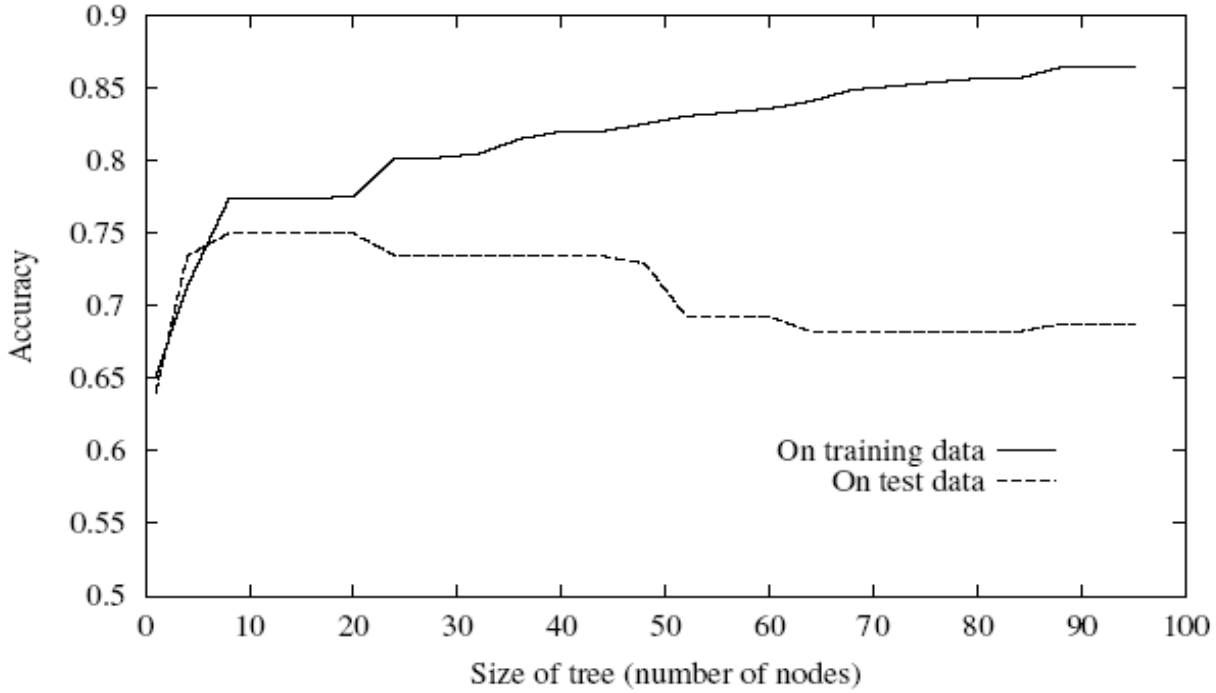
The Gain Ratio measure is defined in terms of the earlier Gain measure, as well as this Split-Information, as follows:

$$GainRatio(S,A) = \frac{Gain(S,A)}{SplitInformation(S,A)}$$

Notice that the Split-information term discourages the selection of attributes with many uniformly distributed values. For example, consider a collection of n examples that are completely separated by attribute A (e.g., Date)

Another decision tree algorithm CART (Classification and Regression Tree) uses the Gini Index method to create split points.

Overfitting (Decision Trees)



Overfitting in decision tree learning. As ID3 adds new nodes to grow the decision tree, the accuracy of the tree measured over the training examples increases monotonically. However, when measured over a set of test examples independent of the training examples, accuracy first increases, then decreases.

Overfitting (Decision Trees)

How can it be possible for tree *h* to fit the training examples better than *h'*, but for it to perform more poorly over subsequent examples?

- **Noise in the training examples**. consider the effect of adding the following positive training example, incorrectly labeled as negative

(Outlook = Sunny, Temperature = Hot, Humidity = Normal, Wind = Strong, PlayTennis = No)

The addition of this incorrect example will now cause ID3 to construct a more complex tree. The result is that ID3 will output a decision tree (*h*) that is more complex than the original tree (*h'*). Of course *h* will fit the collection of training examples perfectly, whereas the simpler *h'* will not.

Overfitting is possible even when the training data are noise-free, it is quite possible for coincidental regularities to occur, in which some attribute happens to partition the examples very well, despite being unrelated to the actual target function.

Reduced Error Pruning

We can use the validation set to prevent overfitting

- Consider each of the decision nodes in the tree to be candidates for pruning.

- Pruning a decision node consists of removing the subtree rooted at that node, making it a leaf node, and assigning it the most common classification of the training examples affiliated with that node.

- Nodes are removed only if the resulting pruned tree performs no worse than the original over the validation set.
  - Any leaf node added due to coincidental regularities in the training set is likely to be pruned because these same coincidences are unlikely to occur in the validation set.

Effect of Reduced Error Pruning

